

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.11

NumPy: Arrays & Indexing

Day 12 — where the loops start disappearing

90 min

Concepts + live code

30 min

Practice notebook

1

New import: numpy as np

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 11

1 Where do you start reading a traceback?

→ The bottom line — it names the error.

2 What is wrong with `except: pass` ?

→ The bug becomes invisible. Count it or log it.

3 Why do specific `except` blocks go first?

→ First match wins, so a general one hides the rest.

4 When does `finally` run?

→ Always — even after a return or an uncaught raise.

5 Why is `from x import *` never right?

→ It silently shadows your own names.

That closes the pure-Python half of Module 1. From today, every session begins with an import.

How today's two hours are structured

1

Why NumPy at all

What is wrong with a plain list

15 min

2

Creating arrays

array, arange, linspace, zeros, ones, random

25 min

3

Indexing & slicing

1-D, 2-D, boolean masks — and views

30 min

4

Reshaping

reshape, flatten, transpose

15 min

5

Mini build

Marks analysis without a single loop

5 min

6

Practice notebook

You code, I walk around

30 min

Section 3 contains the one NumPy behaviour that will surprise you most — a slice is not a copy.

By the end of today you will be able to

1

Create

Build arrays from lists, ranges, and generated values

2

Inspect

Read an array's shape, size and dtype, and know why they matter

3

Select

Index and slice in 1-D and 2-D, and filter with a boolean mask

4

Reshape

Change an array's shape without changing its data

The shift in thinking today

Until now, to do something to every number you wrote a loop. From today you describe the operation and NumPy applies it to the whole array at once. The loops do not just get shorter — they disappear. Getting comfortable with that is worth more than memorising any function name.

1

Why NumPy

What is wrong with a plain list · 15 minutes

- Speed
- Memory
- The ndarray

A list is flexible. That flexibility is the problem.

Doubling a million numbers with a list

```
nums = list(range(1_000_000))

doubled = []
for n in nums:
    doubled.append(n * 2)
```

Python checks the type of every single element, one at a time, a million times. Each number is a separate object scattered across memory.

The same job in NumPy

```
import numpy as np

nums = np.arange(1_000_000)

doubled = nums * 2
```

One line, no loop — and typically 20 to 50 times faster. Every element is the same type, stored in one continuous block, so the work happens in compiled C.

Remember Day 2's warning about nested loops?

Two nested loops over 1,000 items is a million iterations, and three levels is a billion. That slide ended with “this is exactly why NumPy's vectorised operations exist”. This is that moment. The loop has not been made faster — it has been moved out of Python entirely.

The ndarray — one type, one block of memory

A list can hold anything

```
[1, "two", 3.0, True, None]    # all fine
```

An array holds ONE type

```
np.array([1, 2, 3])           dtype: int64  
np.array([1, 2, 3.5])        dtype: float64  <- all promoted
```

What that buys you

- No per-element type checking — the type is known once, for the whole array
- Contiguous memory, so the processor can work on many values at a time
- Operations written in C, not in Python

⚠ The cost of that speed

```
a = np.array([1, 2, 3])  
a[0] = 3.9  
a          -> array([3, 2, 3])    truncated!  
a.append(4) -> AttributeError
```

An int array stores integers, so assigning 3.9 silently truncates it to 3 — no warning at all. And an array has a fixed size: there is no append. If you need to grow a collection, build a Python list first and convert it once at the end.

2

Creating Arrays

Six ways in · 25 minutes

- From a list
- arange & linspace
- zeros, ones, full
- random
- dtype & attributes

Six ways to make an array

Call	Gives	Use it for
<code>np.array([1, 2, 3])</code>	<code>[1 2 3]</code>	data you already have in a list
<code>np.arange(0, 10, 2)</code>	<code>[0 2 4 6 8]</code>	a range — stop is EXCLUDED, like <code>range()</code>
<code>np.linspace(0, 1, 5)</code>	<code>[0. 0.25 0.5 0.75 1.]</code>	N evenly spaced points — stop INCLUDED
<code>np.zeros((2, 3))</code>	a 2x3 of 0.0	a container to fill in later
<code>np.ones((2, 3))</code>	a 2x3 of 1.0	same, or a starting multiplier
<code>np.random.rand(3)</code>	3 values in [0, 1)	test data and simulations



arange excludes, linspace includes

```
np.arange(0, 1, 0.25) -> [0.  0.25 0.5  0.75]
np.linspace(0, 1, 5)  -> [0.  0.25 0.5  0.75 1.  ]
```

Shapes are TUPLES

```
np.zeros((2, 3))    correct - one tuple
np.zeros(2, 3)      TypeError - two arguments
```

What every array can tell you about itself

```
a = np.array([[1, 2, 3],  
             [4, 5, 6]])
```

Attribute	Value	Means
<code>a.shape</code>	<code>(2, 3)</code>	2 rows, 3 columns
<code>a.ndim</code>	<code>2</code>	number of dimensions
<code>a.size</code>	<code>6</code>	total elements
<code>a.dtype</code>	<code>int64</code>	type of every element

shape is the one you will use most

Almost every error you meet in the next six weeks will be a shape mismatch — two things you tried to combine that were not the same size. Printing `.shape` is the first thing to do when something does not work.

```
print(a.shape)    # (2, 3)  
print(b.shape)    # (3, 2)  
a + b  
-> ValueError: operands could not  
    be broadcast together
```

Changing type: `astype` makes a COPY

```
a = np.array([1.7, 2.9])  
a.astype(int)    -> [1 2]    truncated, new array  
np.array([1, 2], dtype=float) # set it up front
```

`astype` returns a new array and leaves the original alone — unlike almost everything else today. Setting `dtype` at creation is usually cleaner than converting afterwards, and it makes your intent obvious to a reader.

3

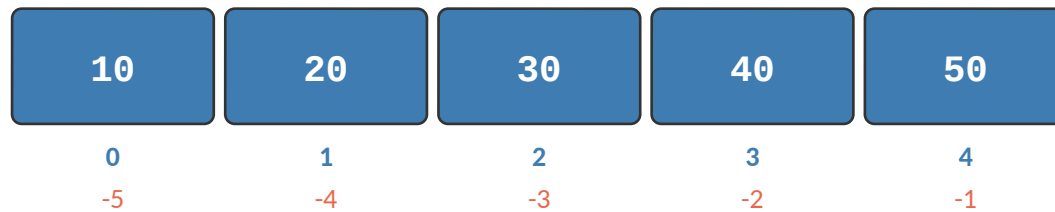
Indexing & Slicing

Including the one that surprises everyone · 30 minutes

- 1-D
- Views not copies
- 2-D
- Boolean masks
- Fancy indexing

1-D indexing works exactly like a list

```
a = np.array([10, 20, 30, 40, 50])
```



```
a[0]      -> 10
a[-1]     -> 50
a[1:4]    -> [20 30 40]
a[::2]    -> [10 30 50]
a[::-1]   -> [50 40 30 20 10]
```

Everything from Day 4 carries over unchanged

Zero-based positions, negative indexes from the end, start:stop:step with the stop excluded, and [::-1] to reverse. You already know all of it — which is why this slide is short and the next one is not.

```
a[7]      -> IndexError: index 7 is out of bounds for axis 0 with size 5
a[1:99]   -> [20 30 40 50]          slicing still never raises, exactly as with a list
```

A NumPy slice is a VIEW, not a copy

This is NOT how lists behave

```
lst = [1, 2, 3, 4]
part = lst[1:3]      # a COPY
part[0] = 99
lst    -> [1, 2, 3, 4]    unchanged
```

```
arr = np.array([1, 2, 3, 4])
part = arr[1:3]      # a VIEW
part[0] = 99
arr    -> [ 1 99  3  4]    CHANGED!
```

Why NumPy does this

Copying a slice of a 10 GB array would mean copying gigabytes of data. A view is just a new window onto the same memory — it costs nothing and is instant.

When you want a real copy, say so:

```
part = arr[1:3].copy()

arr.base is None      # True for an original
part.base is arr      # True for a view
```

The sixth appearance of one idea

Day 1's `c = a` • Day 3's mutable default argument • Day 3's mutable argument • Day 4's `b = a` • Day 7's shared class attribute • and now a NumPy view. Every one is the same sentence: two names, one piece of data. What is new today is that even SLICING now shares — so the habit of writing `.copy()` when you mean a copy matters more here than anywhere else in the course.

Two dimensions: [row, column]

```
m = np.array([[1,2,3], [4,5,6], [7,8,9]])
```

0	1	2	3
1	4	5	6
2	7	8	9

m[1, 1] is 5

```
m[1, 1]    -> 5    preferred
m[1][1]    -> 5    works, but slower
```

Slicing both axes at once

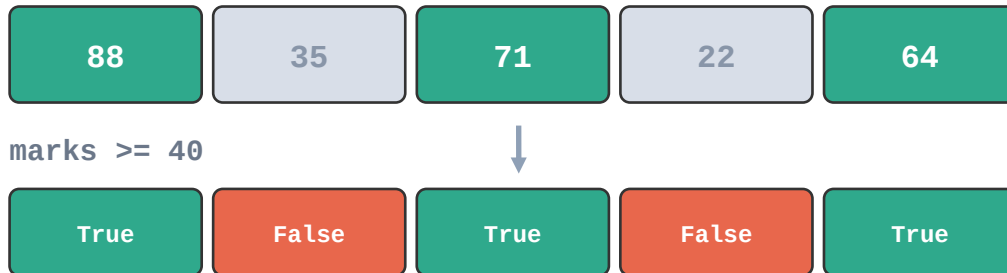
```
m[0]        -> [1 2 3]    first row
m[:, 0]      -> [1 4 7]    first COLUMN
m[0:2, 1:]   -> [[2 3]
                  [5 6]]
m[-1, -1]    -> 9

# the comma separates the axes:
# m[rows, columns]
# a bare : means 'all of this axis'
```

The colon is the part to practise. `m[:, 0]` reads as “every row, column 0” — which is how you pull a single column out of a table. In three sessions that becomes `df["age"]` in pandas, and it is the same idea underneath. If you can read `m[1:3, :2]` aloud correctly, 2-D indexing is done.

Boolean masking — the payoff

```
marks = np.array([88, 35, 71, 22, 64])
```



```
marks >= 40
-> [ True False  True False  True]

marks[marks >= 40]
-> [88 71 64]    only the passes
```

Combining conditions

```
marks[(marks >= 40) & (marks < 80)]
-> [71 64]

# & is and      | is or      ~ is not
# BRACKETS around each condition are required
```

⚠ Not and / or

```
marks[(marks >= 40) and (marks < 80)]
-> ValueError: truth value of an array
    with more than one element is ambiguous
```

Python's and/or want a single True or False. An array of five booleans is neither, so NumPy refuses to guess.

4

Reshaping

Same data, different shape · 15 minutes

- reshape
- The -1 trick
- flatten & ravel
- transpose

Changing shape without changing data

`a = np.arange(6)`

`a.reshape(2, 3)`



shape (6,)



shape (2, 3) — same six numbers, read row by row

```
a.reshape(2, 3)    rows x columns
a.reshape(3, 2)    also fine
a.reshape(4, 2)    ValueError - 6 != 8
```

The -1 trick

```
a.reshape(2, -1)    -> (2, 3)    work it out
a.reshape(-1, 1)    -> (6, 1)    a column
a.reshape(-1)       -> (6,)      flat again

# -1 means 'you figure out this one'
```

Flattening and transposing

```
m.flatten()    1-D, always a COPY
m.ravel()      1-D, a VIEW when it can be
m.T           transpose: rows become columns

# reshape also returns a VIEW when possible
```

Debug corner — eight NumPy bugs

You wrote	What happens	Fix
<code>np.zeros(2, 3)</code>	<code>TypeError</code>	<code>np.zeros((2, 3))</code>
<code>part = arr[1:3]</code>	changing part changes arr	<code>arr[1:3].copy()</code>
<code>arr.append(4)</code>	<code>AttributeError</code> — fixed size	build a list, convert at the end
<code>a[0] = 3.9</code> on an int array	silently stores 3	<code>dtype=float</code> at creation
<code>(x > 1) and (x < 5)</code>	<code>ValueError: ambiguous</code>	<code>(x > 1) & (x < 5)</code>
<code>x > 1 & x < 5</code>	wrong answer — <code>&</code> binds tighter	bracket each condition
<code>m[1][1]</code>	works, but slower	<code>m[1, 1]</code>
<code>a.reshape(4, 2)</code> on 6 items	<code>ValueError</code> — size must match	<code>a.reshape(2, -1)</code>

Row 2 is the one that will cost you an afternoon. All eight are in today's practice notebook.

Putting it together — marks analysis, no loops

```
import numpy as np

# 6 students x 3 subjects
marks = np.array([[88, 71, 64],
                  [91, 84, 79],
                  [45, 38, 52],
                  [67, 73, 70],
                  [95, 88, 92],
                  [30, 41, 35]])

print("shape      :", marks.shape)      # (6, 3)

student_avg = marks.mean(axis=1)        # across COLUMNS
subject_avg = marks.mean(axis=0)        # down ROWS

print("per student :", student_avg.round(1))
print("per subject :", subject_avg.round(1))

passed = student_avg >= 40               # a boolean mask
print("pass count  :", passed.sum())     # True counts as 1
print("top average :", student_avg.max().round(1))

python_marks = marks[:, 0]              # first column
print("python > 80  :", python_marks[python_marks > 80])
```

**One array, no loops**

six rows handled at once

**axis=1**

across the columns — per student

**axis=0**

down the rows — per subject

**Boolean mask**

Day 1's >= on a whole array

**.sum() on booleans**

Day 1: True counts as 1

**m[:, 0]**

every row, column 0

**Chained filter**

mask a column you extracted

On Day 2 this would have been four nested loops. Remember `axis=1` as “across” and `axis=0` as “down”.

Practice session — 30 minutes



Predict the output

Creation, dtype, views, masks

8 questions



Spot & fix the bug

Shape tuples, views, and/or, reshape

4 questions



Write the code

Build, slice, filter, reshape, columns

5 tasks

Your notebook

[Day12_Practice_Questions.ipynb](#)

- Print `.shape` when stuck
- Ask: view or copy?
- Raise a hand — I am walking around
- Solutions shared after class

If an array changed and you did not expect it to, you were holding a view. If shapes will not combine, print both `.shape` values first.

TODAY IN TWELVE LINES

Recap

- 1 `import numpy as np` — always that alias.
- 2 An array holds ONE type, in one block of memory.
- 3 That is why it is fast, and why it cannot grow.
- 4 `np.zeros((2, 3))` — the shape is a TUPLE.
- 5 `arange` excludes the stop; `linspace` includes it.
- 6 `shape`, `ndim`, `size`, `dtype` describe any array.
- 7 A slice is a VIEW — changing it changes the original.
- 8 Use `.copy()` when you actually want a copy.
- 9 `m[row, column]`; a bare `:` means all of that axis.
- 10 `m[:, 0]` is a column; `m[0]` is a row.
- 11 Masks filter without a loop. Use `&` | `~`, not and or not.
- 12 `reshape` keeps the data; `-1` means 'work it out'.

Homework

Rewrite Day 2's times table with `np.arange` and `reshape` · Take a 5x4 array and print each row's max without a loop · Prove a slice is a view, then break the link with `.copy()`

Next class — Topic 1.12

Broadcasting & vectorised operations: arithmetic between arrays of different shapes, and why the loop disappears entirely